Lecture 7 · Machine-Level Programming IV: Data

10/15 (목) · 슬라이드 "Lecture 7 Data" + 수업 녹취. 슬라이드 14~35(multi-level 배열, N×N 행렬, struct, alignment, floating point)는 녹화가 없어 슬라이드만 보고 10/17에 추가 정리

1 데이터 파트의 시작

오늘의 범위: array 먼저, struct는 그다음

지금까지는 기본 data type을 하나씩만 다뤘음: `char` 1byte, `short` 2byte, `int` 4byte, `long` · 포인터· `double` 8byte. 이제는 같은 종류의 값을 모아 저장하는 **array**와 서로 다른 type을 묶는 **struct**를 다룸.

- 아키텍처가 하는 일은 최소한: 64bit register를 통째로, 절반, 1/4로 쓸 수 있어서 8·4·2·1 byte 기본 type을 표현할 수 있을 뿐
- "int 배열"을 직접 표현하는 하드웨어 기능은 없음
- 배열 접근은 전부 **offset 계산**: 앞에서 배운 lea의 주소 지정 모드(base, index, scale, displacement)로 특정 index의 주소를 계산
- 컴파일러는 배열 인덱스를 base·index·scale·displacement를 이용한 주소 계산으로 변환한다. 그러나 일반적인 C에서는 배열 범위 검사를 자동으로 보장하지 않으므로, 유효한 인덱스를 사용하는 것은 프로그래머의 책임이다.
- 결국 "base 주소가 어디이고 offset이 얼마인가"로 귀결 → 포인터/주소 산술로 1차원 배열, 다차원 배열, struct에 접근하는 방법을 배움
- 슬라이드 2 agenda: Arrays (One-Dimensional → Multi-Dimensional(Nested) → Multi-Level) → Structures (Allocation, Access, Alignment) → Floating Point

2 1차원 배열: 할당과 포인터 산술

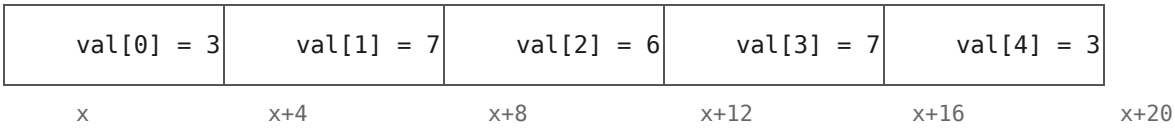
배열 할당: T N[L]

$T\ N[L]$: data type T, 길이 L, 이름 N → 메모리에 연속된 $L \times \texttt{sizeof(T)}$ byte 할당 · 원소 i의 주소 = $N + \texttt{sizeof(T)} \cdot i$

- 시작 주소를 x라 하면 (슬라이드 예): `char string[12]` → 1 × 12 = 12byte (주소 $x, x+1, \ldots, x+11$) · `int val[5]` → 20byte ($x, x+4, \ldots, x+16$) · `double a[3]` → 24byte ($x, x+8, x+16$) · `char *p[3]` → 포인터 8 × 3 = 24byte
- 원소 간격이 일정(= 원소 크기)해서 특정 원소의 위치를 곱셈 한 번, 덧셈 한 번으로 계산할 수 있음

배열 접근과 byte 배치: little endian

int val[5] = {3, 7, 6, 7, 3} — 시작 주소 x, 원소 4byte



x x+4 x+8 x+12 x+16 x+20

val[1]의 4byte를 byte 단위로 펼치면 (little endian)

07	00	00	00

← least significant byte가 가장 낮은 주소 x+4에 놓임

메모리에는 03 00 00 00 · 07 00 00 00 · 06 00 00 00 · ⋯ 순서로 저장. Intel이 little endian이라 03이 맨 왼쪽(낮은 주소)에 옴

배열 이름과 포인터 산술

식	type	값	의미
`val[4]`	int	3	index로 값을 읽음 (마지막 원소)
`val[5]`	int	??	범위 밖: 주소는 $x+20$, 값은 알 수 없음 (C에서 읽으면 UB)
`val`	int *	x	배열 이름 자체가 첫 원소의 주소
`val+1`	int *	$x+4$	정수를 더하면 원소 크기를 곱해 주소가 늘어남
`&val[2]`	int *	$x+8$	2번 원소의 주소
`*(val+1)`	int	7	주소 계산 뒤 역참조 → `val[1]`의 값
`val+i`	int *	$x+4i$	일반형: 주소 = 시작 주소 + 원소 크기 × index

- `val[i]` 는 `*(val+i)` 와 같은 뜻
- int 배열이라 +1이 4byte, double 배열이면 8byte씩 움직임

범위 검사는 없다

C와 하드웨어는 배열 범위를 검사하지 않음. `val[5]` 는 배열 바로 다음 칸($x + 20$)을 읽지만 무엇이 들어 있는지 알 수 없음 — C 표준상 범위 밖 접근은 정의되지 않은 동작(UB) (one-past 포인터 val+5까지는 만들 수 있지만 역참조는 UB).

3 zincr: 배열을 도는 어셈블리

```
#define ZLEN 5
typedef int zip_vec[ZLEN];
zip_vec postech = {3, 7, 6, 7, 3};

void zincr(zip_vec z) {        // int z[5]와 같은 선언 (typedef로 만든 type 이름)
    size_t i;
    for (i = 0; i < ZLEN; i++)
        z[i]++;              // 모든 원소를 1씩 증가 → 새 zip code는 4 8 7 8 4
}
```

```
# z in %rdi
    movl $0, %eax            # %eax = i
    jmp .L3
.L4:                          # loop:
    addl $1, (%rdi,%rax,4)    # z[i]++
    addq $1, %rax             # i++
.L3:                          # middle
    cmpq $4, %rax             # compare i and 4
    jbe .L4                   # if <=, goto loop
    rep; ret
```

- z는 첫 번째 인자 → %rdi (배열의 첫 byte 주소), %rax가 반복 변수 i
- 구조는 Lec5의 jump-to-middle: 처음에 조건 검사(.L3)로 점프하고, 참이면 .L4 body로 돌아옴
- `cmpq $4, %rax` + `jbe` : ZLEN = 5라서 i < 5가 i ≤ 4로 컴파일됨
- **size_t는 unsigned라 signed용 jle가 아니라 unsigned용 jbe**
- `addl $1, (%rdi,%rax,4)` : 주소 $\%rdi + 4 \cdot \%rax$의 메모리 값에 1을 더해 그 자리에 씀 → z[i]++
- base = 배열 시작(%rdi), index = i(%rax), scale = 원소 크기(4). 컴파일러가 type이 int임을 알기 때문에 scale을 4로 정함

손계산: zincr 추적 (%rdi = 16, postech)

i	cmpq 결과 (i vs 4)	jbe	addl 대상 주소 = 16 + 4i	값 변화
0	i < 4 (CF=1)	점프	16	3 → 4
1	i < 4 (CF=1)	점프	20	7 → 8
2	i < 4 (CF=1)	점프	24	6 → 7
3	i < 4 (CF=1)	점프	28	7 → 8
4	i = 4 (ZF=1)	점프	32	3 → 4
5	i > 4 (CF=0, ZF=0)	안 함	없음	ret

결과: 3 7 6 7 3 → 4 8 7 8 4 (수업에서 말한 새 zip code와 일치). i = 5에서 5 − 4 = 1 → borrow 없음 (CF = 0), 0도 아님(ZF = 0) → jbe(CF = 1 또는 ZF = 1)가 거짓이라 loop 종료. body는 5번, 비교는 6번 실행 (jump-to-middle이라 마지막 실패 비교가 한 번 더).

4 중첩(다차원) 배열: row-major

- `T N[R][C];` : data type T, R개 행, C개 열. 첫 번째 index가 행, 두 번째 index가 열
- 원소 개수 $R \times C$, 원소 하나가 $k =$ `sizeof(T)` byte → 전체 크기 $R \cdot C \cdot k$ byte (int면 $4RC$)
- 메모리는 byte의 1차원 배열 → 2차원 행렬은 가상의 표현이고 실제로는 일렬로 펴서 저장
- 펴는 방법 두 가지. row-major가 가장 널리 쓰이고 이 수업에서 사용:

방식	순서	2×3 예
row-major ordering	첫 행의 열 원소들을 연속으로, 그다음 둘째 행, …	A00 A01 A02 A10 A11 A12
column-major ordering	첫 열의 행 원소들을 연속으로, …	A00 A10 A01 A11 A02 A12

- 슬라이드도 중첩 배열은 row-major 배치가 보장된다고 함

행 i의 시작 주소 = $N + i \cdot (C \cdot k)$ · 원소 $N[i][j]$의 주소 =
$N + i \cdot (C \cdot k) + j \cdot k = N + (i \cdot C + j) \cdot k$

int pohang[4][5] (시작 주소 76) — row-major로 편 모습, 한 행 20byte

3 7 6 7 0	3 7 6 7 1	3 7 6 7 3	3 7 6 7 4
76 (행 0)	96 (행 1)	116 (행 2)	136 (행 3)

행 간격은 늘 20 (= 5 × 4)
전체 80byte → 주소 76~155

pohang[2][3]의 주소 계산

행 2의 시작으로 이동: 76 + 2 × 20 = 116 → 거기서 열 3만큼 더: 116 + 3 × 4 = 128
한 식으로: 76 + 2·(5·4) + 3·4 = 76 + (2·5 + 3)·4 = 76 + 52 = 128

먼저 목표 행으로 이동한 뒤 목표 열만큼 조금 더 이동하는 두 단계. 어셈블리는 이 둘을 index 하나와 scale 4로 합쳐 처리

```
#define PCOUNT 4
zip_vec pohang[PCOUNT] = {{3, 7, 6, 7, 0},   // zip_vec pohang[4]는
                          {3, 7, 6, 7, 1},   // int pohang[4][5]와 같은 선언
                          {3, 7, 6, 7, 3},
                          {3, 7, 6, 7, 4}};  // 전체 원소가 row-major 순서로 놓이는 것이 보장됨
```

행 접근: get_pohang_zip

```
int *get_pohang_zip(int index) {
    return pohang[index];       // int 5개짜리 배열 → 그 행의 시작 주소를 반환
}
```

```
# index in %rdi
    leaq (%rdi,%rdi,4), %rax    # 5 * index
    leaq pohang(,%rax,4), %rax  # pohang + (20*index)
    ret
```

- 행 시작 주소 = $\text{pohang} + 20 \cdot \text{index}$ (슬라이드의 "pohoang"은 pohang 오타)
- 첫 `leaq (%rdi,%rdi,4)` : %rdi + 4·%rdi = 5 × index. 행 하나가 int 5개라 5를 곱함
- **scale에는 1, 2, 4, 8만 쓸 수 있어서 5배는 index + 4×index로 만듦** (곱셈 대신 lea 한 번)
- 둘째 `leaq pohang(,%rax,4)` : displacement = pohang(배열 label), index = %rax(= 5·index), scale 4(int 크기)

- 둘 다 leaq라 메모리를 읽지 않고 주소만 계산해 반환
- 검산 (pohang = 76): index 0→76, 1→76+20=96, 2→116, 3→136. get_pohang_zip(3): %rax = 15 → 76 + 60 = 136 → 마지막 행(3 7 6 7 4, 주소 136~155)의 시작

원소 접근: get_pohang_digit

```
int get_pohang_digit(int index, int col) {
    return pohang[index][col];  // 슬라이드는 반환형이 int *로 적혀 있으나 원소 값을 반환하므로 int
}
```

```
# index in %rdi, col in %rsi
    leaq (%rdi,%rdi,4), %rax    # 5*index
    addq %rax, %rsi             # 5*index+col  (슬라이드 표기 addl — index·col이 32bit int
    movl pohang(,%rsi,4), %eax  # M[pohang+4*(5*index+col)]
    ret
```

- 주소 = $\text{pohang} + 4 \cdot (5 \cdot \text{index} + \text{col})$
- 첫 leaq로 행 위치 5·index
- col을 더해 "1차원으로 폈을 때의 index" 5·index + col
- movl에서 원소 크기 4를 곱한 주소의 메모리를 읽어 %eax에 저장. 이번에는 movl이라 메모리를 실제로 읽음

손계산: get_pohang_digit(2, 3) (pohang = 76)

① `leaq (%rdi,%rdi,4), %rax` → %rax = 2 + 4·2 = 10

② `addq %rax, %rsi` → %rsi = 3 + 10 = 13 (슬라이드 표기 addl — index·col이 32bit int라 5·index+col이 32bit 범위를 넘지 않는 이 예에서는 addq와 결과 동일; 32bit 쓰기는 상위 32bit를 0으로 만듦)

③ `movl pohang(,%rsi,4), %eax` → 주소 = 76 + 4·13 = 76 + 52 = 128 → %eax = M[128]

④ 검산: 행 2의 시작 116 + 열 3 × 4 = 116 + 12 = 128 → 같은 주소. 식으로도
$76 + 2 \cdot 20 + 3 \cdot 4 = 128$

⑤ flat하게 나열한 20개에서 index 10~14가 3 7 6 7 3 → index 13은 7 → %eax = 7 (슬라이드 그림에서 파란 행의 네 번째 칸)

다른 칸도 같은 식: pohang[0][0] → 76 (값 3) · pohang[1][4] → 76 + 20 + 16 = 112 (값 1) · pohang[3][4] → 76 + 60 + 16 = 152 (값 4) → 152 + 4 = 156 = 76 + 80 (배열 끝 다음 주소)

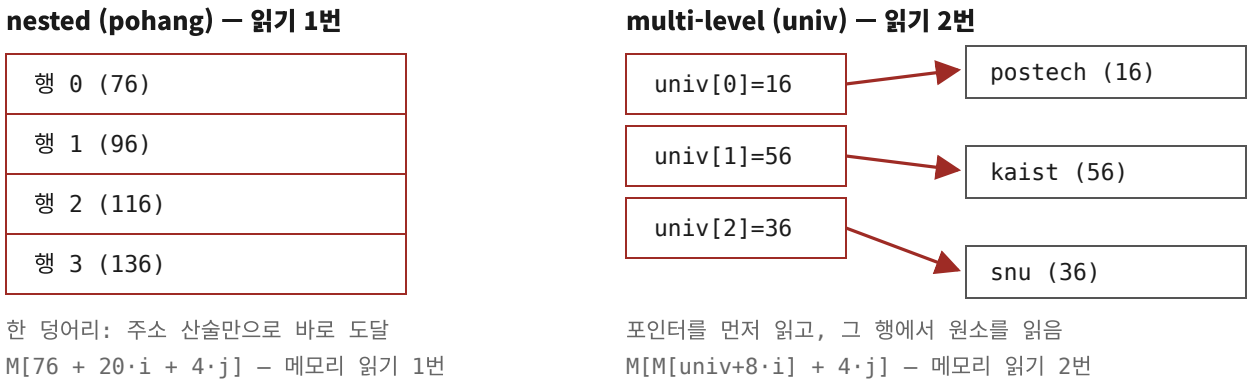
5 다단계(multi-level) 배열

```
zip_vec postech = {3, 7, 6, 7, 3};
zip_vec snu     = {0, 8, 8, 2, 0};
zip_vec kaist   = {3, 4, 1, 4, 1};

#define UCOUNT 3
int *univ[UCOUNT] = {postech, kaist, snu};
```

- univ는 포인터(8byte) 원소 3개짜리 배열 → 각 포인터가 int 5개짜리 배열을 가리킴
- 앞의 nested 배열 pohang은 5개짜리 행 4개가 **한 덩어리**로 붙어 있었음. 여기서는 행 3개가 따로 선언되어 있고 univ는 그 행들의 시작 주소만 모아 둔 배열
- 슬라이드 가정 (univ의 시작 주소 160, 원소 간격 8byte):
  - univ[0] = 16 → postech (16~35)
  - univ[1] = 56 → kaist (56~75, 초기화 순서가 {postech, kaist, snu}라 둘째 칸이 kaist)
  - univ[2] = 36 → snu (36~55)

- 행 시작 주소가 16, 56, 36으로 규칙 없이 흩어져 있음. nested 배열의 행 시작이 늘 $C \cdot k$ 간격으로 일정했던 것과 다른 점
- 포인터만 저장하므로 각 행이 메모리 어디에 있어도 되고 **행마다 길이가 달라도 됨**
- 대신 원소를 읽을 때 포인터를 한 번 따라가야 함



같은 "행·열" 접근인데 도달 방법이 다름. nested는 주소 계산 한 번, multi-level은 포인터 chase가 추가됨

```
int get_univ_digit(size_t index, size_t col) {
    return univ[index][col];     // 슬라이드는 col을 dig(digit)이라고 부름
}
```

```
# index in %rdi, col in %rsi
    salq $2, %rsi                # 4*dig
    addq univ(,%rdi,8), %rsi     # p = univ[index] + 4*dig
    movl (%rsi), %eax            # return *p
    ret
```

- `salq $2, %rsi` : col을 왼쪽으로 2칸 shift → 4·col (원소 int 크기 4를 곱한 열 offset)
- `addq univ(,%rdi,8), %rsi` : 메모리 univ + 8·index에서 행 포인터 univ[index]를 읽어 열 offset에 더함 → p. scale 8은 포인터 크기 (univ의 원소가 포인터라서)
- 이 한 명령이 메모리 읽기(첫 번째)와 덧셈을 함께 함
- `movl (%rsi), %eax` : p가 가리키는 int를 읽어 반환 (두 번째 메모리 읽기)
- 슬라이드 정리: 접근 식은 `Mem[Mem[univ+8*index]+4*dig]` → **메모리 읽기가 두 번** (첫 번째는 목표 행 배열의 포인터, 두 번째는 그 배열 안의 목표 원소)

손계산: nested와 multi-level 원소 접근 비교 (index = 1, col = 2)

① nested pohang[1][2] (pohang = 76, 행 하나 20byte): 주소 = 76 + 20·1 + 4·2 = 104 → 메모리 읽기 1번: M[104] = 6 (행 1 = 3 7 6 7 1의 셋째 값)

② multi-level get_univ_digit(1, 2) (univ = 160): salq → %rsi = 8 · addq → 160 + 8·1 = 168에서 M[168] = 56 (첫 번째 읽기) → %rsi = 8 + 56 = 64 · movl → M[64] = 1 (두 번째 읽기, kaist의 셋째 값)

③ univ[2][4]도 같은 순서: M[160 + 16] = M[176] = 36 → 36 + 4·4 = 52 → M[52] = 0 (snu의 마지막 값)

④ 같은 열 offset 8(= col 2)이어도 행 포인터가 달라 도착하는 곳이 달라짐: univ[0] → 16 + 8 = 24 (값 6) · univ[1] → 56 + 8 = 64 (값 1) · univ[2] → 36 + 8 = 44 (값 8)

최종 답

nested: 주소를 산술로만 구해 읽기 1번 · multi-level: 행 포인터를 읽고 원소를 읽어 2번

6 N×N 행렬: 고정 크기 vs 가변 크기

행렬 코드의 세 가지 형태

```
// ① Fixed dimensions — 컴파일 시점에 N을 앎
#define N 16
typedef int fix_matrix[N][N];
int fix_ele(fix_matrix a, int i, int j) { return a[i][j]; }

// ② Variable dimensions, explicit indexing — 동적 배열의 전통적 방법
#define IDX(n, i, j) ((i)*(n)+(j))
int vec_ele(size_t n, int *a, int i, int j) { return a[IDX(n,i,j)]; }

// ③ Variable dimensions, implicit indexing — 지금은 gcc가 지원
int var_ele(size_t n, int a[n][n], int i, int j) { return a[i][j]; }
// i*n+j 계산을 컴파일러가 해 줌. 인자 n이 a보다 앞에 와야 a[n][n]에서 쓸 수 있음
```

16×16 행렬 접근 (고정 크기)

```
# a in %rdi, i in %rsi, j in %rdx
    salq $6, %rsi             # 64*i
    addq %rsi, %rdi           # a + 64*i
    movl (%rdi,%rdx,4), %eax  # M[a + 64*i + 4*j]
    ret
```

- 원소 주소 $a + i \cdot (C \cdot k) + j \cdot k$에서 C = 16, k = 4 → 한 행이 64byte = 2^6
- `salq $6, %rsi` : 왼쪽으로 6칸 shift = 64배 → 행 시작까지의 offset
- **N이 컴파일 시점의 상수라 행 크기도 상수 → 곱셈 없이 shift와 덧셈만으로 주소가 나옴**

---

n×n 행렬 접근 (가변 크기)

```
# n in %rdi, a in %rsi, i in %rdx, j in %rcx
    imulq %rdx, %rdi          # n*i
    leaq (%rsi,%rdi,4), %rax  # a + 4*n*i
    movl (%rax,%rcx,4), %eax  # a + 4*n*i + 4*j
    ret
```

- n이 실행 중에 정해짐 → 정수 곱셈(`imulq`)이 필요: %rdi = n × i (행 번호 × 열 개수)
- 16×16과 비교: 행 크기 64가 상수일 때는 `salq $6` , 행 크기 4·n이 변수일 때는 `imulq`

손계산: 16×16과 n×n의 원소 주소 (a = 0x1000, i = 3, j = 5)

① fix_ele(a, 3, 5): salq → %rsi = 3 ≪ 6 = 192 · addq → %rdi = 0x1000 + 192 = 0x10C0 · movl → 주소 0x10C0 + 4·5 = 0x10D4

② var_ele(16, a, 3, 5): imulq → %rdi = 48 · leaq → %rax = 0x1000 + 4·48 = 0x10C0 · movl → 주소 0x10D4

③ 검산: (3 × 16 + 5) × 4 = 212 = 0xD4 → 0x1000 + 0xD4 = 0x10D4. 두 함수가 같은 주소를 읽음

④ n = 10이면: imulq로 10 × 3 = 30 → 0x1000 + 120 = 0x1078 → 열 offset 20을 더해 0x108C. 검산: (3 × 10 + 5) × 4 = 140 = 0x8C ✓

최종 답 **주소는 둘 다 a + 4·n·i + 4·j. 행 크기가 상수여야 곱셈을 shift로 바꿀 수 있음**

배열 주소 계산 — Array Indexing

중첩 배열(nested) | 다단계 배열(multi-level)

배열 선택 — int A[4][5]

A[0][0]	A[0][1]	A[0][2]	A[0][3]	A[0][4]
A[1][0]	A[1][1]	A[1][2]	A[1][3]	A[1][4]
A[2][0]	A[2][1]	A[2][2]	A[2][3]	A[2][4]
A[3][0]	A[3][1]	A[3][2]	A[3][3]	A[3][4]

연속 저장 — Row-major (80 Bytes)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79

```
선택: A[1][2]

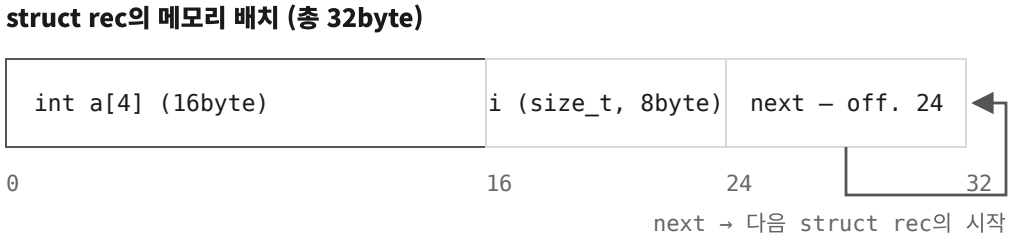
주소 = base + (i × C + j) × sizeof(int)
     = 0x0000000000001000 + (1 × 5 + 2) × 4
     = 0x0000000000001000 + 28
     = 0x000000000000101C

sizeof(A) = 4 × 5 × 4 = 80 B
int* p의 p+1은 4바이트 전진합니다.
int (*)[5] p의 p+1은 20바이트 전진합니다.
중첩 배열은 모든 행을 연속 저장합니다.
```

7 struct: 표현과 멤버 접근

```
struct rec {
    int a[4];
    size_t i;
    struct rec *next;
};
```

- 구조체는 모든 field를 담을 만큼 큰 메모리 block 하나로 표현됨
- field는 **선언한 순서대로** 배치됨. 다른 순서가 더 compact한 표현을 만들 수 있어도 순서를 바꾸지 않음
- 전체 크기와 field 위치는 컴파일러가 정함 → **기계어 수준 프로그램은 소스의 구조체를 이해하지 못함 (남는 것은 offset 숫자뿐)**
- 크기: a 4 × 4 = 16 @0, i(size_t) 8 @16, next(포인터) 8 @24 → 16 + 8 + 8 = 32byte (padding 없음)
- next는 다음 struct rec의 시작을 가리키는 포인터



field는 선언 순서대로: a는 0부터, i(8byte)는 16~23, padding 없이 next는 24부터

멤버 포인터 생성: get_ap

```
int *get_ap(struct rec *r, size_t idx) {
    return &r->a[idx];    // 각 member의 offset은 컴파일 시점에 결정 → 주소는 r + 4*idx
}
```

```
# r in %rdi, idx in %rsi
    leaq (%rdi,%rsi,4), %rax   # base = r, index = idx, scale = 4 (a의 offset이 0)
    ret                        # leaq 하나 — 메모리를 읽지 않고 주소만 계산
```

---

set_val: next를 따라가며 a[i]에 쓰기

```
void set_val(struct rec *r, int val) {
    while (r) {
        int i = r->i;
        r->a[i] = val;
        r = r->next;
    }
}
```

```
.L11:                          # loop:  (%rdi = r, %rsi = val)
    movslq 16(%rdi), %rax      # i = Mem[r+16]
    movl %esi, (%rdi,%rax,4)   # Mem[r+4*i] = val
    movq 24(%rdi), %rdi        # r = Mem[r+24]
    testq %rdi, %rdi           # Test r
    jne .L11                   # if !=0 goto loop
```

- member 접근이 전부 r에 offset을 더하는 꼴:
  - `16(%rdi)` = r->i (offset 16)
  - `24(%rdi)` = r->next (offset 24)
  - `(%rdi,%rax,4)` = r->a[i] (offset 0 + 4·i)
- r->i는 8byte size_t인데 int i에 담으므로 아래쪽 4byte만 `movslq`로 읽어 64bit로 부호 확장
- `testq + jne`: r이 0이 아니면 다음 node로 loop 계속, 0이면 종료. 슬라이드에는 loop 몸체만 나옴

손계산: set_val 추적 (node 0x1000: i = 1, next = 0x2000 / node 0x2000: i = 3, next = 0, val = 9)

① 반복 1 (%rdi = 0x1000): movslq → M[0x1010] = 1 → %rax = 1 · movl → M[0x1000 + 4] = M[0x1004] = 9 · movq → M[0x1018] = 0x2000 → %rdi = 0x2000 · testq → 0 아님 → .L11로 점프

② 반복 2 (%rdi = 0x2000): movslq → M[0x2010] = 3 → %rax = 3 · movl → M[0x200C] = 9 · movq → M[0x2018] = 0 → %rdi = 0 · testq → 점프 안 함, 종료

최종 답

첫 node의 a[1]과 둘째 node의 a[3]이 9로 바뀜. 각 node에서 어느 칸을 쓸지는 그 node의 i가 정함

8 struct alignment

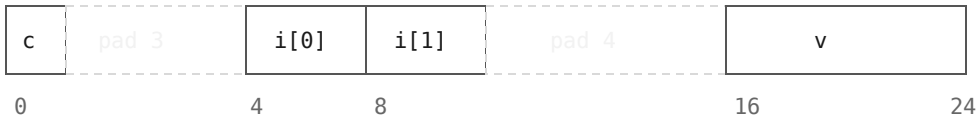
정렬 원칙

- **Aligned data**: 기본 data type이 B byte를 요구하면 주소는 B의 배수. 일부 machine에서는 필수, x86-64에서는 권장
- type별 요구 B (= type 크기): char 1 · short 2 · int·float 4 · long·double·포인터 8
- 정렬하는 동기:
  - memory는 (시스템에 따라) 4byte 또는 8byte 단위의 정렬된 chunk로 접근됨
  - quad-word 경계에 걸친 data의 load/store는 비효율
  - virtual memory에서 data가 두 page에 걸치면 더 까다로움
- compiler가 구조체 안에 gap(padding)을 넣어 각 field의 정렬을 맞춤

구조체 안: 각 element의 정렬 요구 충족 · 구조체 전체: K = element들 중 가장 큰 정렬 요구 → 구조체의 시작 주소와 전체 길이가 모두 K의 배수

S1: internal padding

struct S1 { char c; int i[2]; double v; } — K = 8, sizeof = 24



i[0]은 4의 배수인 4로, v는 8의 배수인 16으로 올림 → padding 두 군데 모두 구조체 안쪽 (internal padding)
정렬 없이 붙이면 c p+0, i[0] p+1, i[1] p+5, v p+9, 끝 p+16 (17byte) → 17은 K = 8의 배수가 아니라 전체 길이 조건도 불만족

시작 주소 p는 8의 배수여야 함 (슬라이드 그림의 "Multiple of 8"은 c가 아니라 구조체 시작 p에 대한 조건)

S2: 멤버 순서만 바꾸면 padding 위치가 달라짐

```
struct S2 {              // S1과 멤버는 같고 순서만 다름
    double v;
    int i[2];
    char c;
} *p;
```

- 그림: v가 p+0, i[0]이 p+8, i[1]이 p+12, c가 p+16, 그 뒤 회색 칸이 끝(p+23)까지 → 전체 24byte, 8의 배수
- c 뒤에서 끝내면 17byte라 8의 배수가 아니므로 뒤쪽에 7byte를 채움 (**trailing padding**). 가장 큰 정렬 요구가 K이면 구조체 전체 길이가 K의 배수여야 하기 때문

손계산: S1, S2의 offset과 sizeof

규칙: 멤버 offset은 이전 멤버 끝을 그 멤버의 정렬(B)의 배수로 올린 값, 전체 크기는 마지막 끝을 K의 배수로 올린 값.

S1 (K = 8)	크기	정렬	이전 끝	offset 결정	끝
c	1	1	0	0	1
i[0]	4	4	1	4의 배수로 올림 → 4	8
i[1]	4	4	8	8	12
v	8	8	12	8의 배수로 올림 → 16	24

전체 끝 24 → 8의 배수 → sizeof = 24 · padding: p+1~3 (3byte) + p+12~15 (4byte) = 7byte · 검산: 멤버 크기 1 + 8 + 8 = 17, 24 − 17 = 7 ✓

S2 (K = 8)	offset	끝
v	0	8
i[0]	8	12
i[1]	12	16
c	16	17

전체 끝 17 → 8의 배수로 올림 → sizeof = 24 · padding: p+17~23 (7byte, 전부 끝쪽)

최종 답

S1과 S2는 멤버가 같고 순서만 달라도 크기는 둘 다 24. 줄어드는 것은 없고 padding 7byte의 위치만 달라짐 (S1은 안쪽 3 + 4, S2는 끝쪽 7)

9 struct 배열과 공간 절약

struct 배열

- `struct S2 a[10];` : 구조체 전체 길이는 K의 배수, 그리고 배열의 모든 원소도 정렬 요구 충족
- a[0]은 a, a[1]은 a+24, a[2]는 a+48 (원소 하나 24byte)
- **끝쪽 padding 덕분에 다음 원소의 v도 8의 배수에 놓임**. 끝 padding이 없으면 원소 크기가 17이 되어 a[1]의 v가 a+17에 놓여 8의 배수가 아님
- a[1]을 확대하면: v가 a+24, i[0]이 a+32, c가 a+40, 마지막 byte가 a+47

배열 원소 접근: get_j

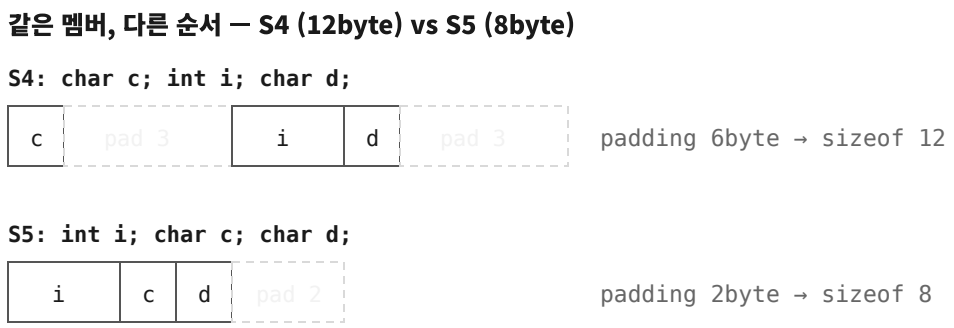
```
struct S3 {
    short i;
    float v;
    short j;
} a[10];

short get_j(int idx) {
    return a[idx].j;
}
```

```
# idx in %rdi
    leaq (%rdi,%rdi,2), %rax    # 3*idx
    movzwl a+8(,%rax,4), %eax
    ret
```

- sizeof(S3) = 12. alignment spacer(padding)를 포함한 크기. 배열 offset은 12·idx, member j는 구조체 안 offset 8
- `leaq (%rdi,%rdi,2)` : idx + 2·idx = 3·idx. **12배는 scale에 쓸 수 없어서(1, 2, 4, 8뿐) 3배를 먼저 만들고 movzwl의 scale 4로 12·idx를 완성**
- `movzwl a+8(,%rax,4), %eax` : 주소 a + 8 + 4·(3·idx) = a + 12·idx + 8에서 short 2byte를 읽어 0으로 채워 %eax에 (movz = zero extend, w = 2byte 원본, l = 4byte 목적지)
- a+8의 offset 값은 linking 때 정해짐

공간 절약: 큰 data type을 앞에



큰 data type을 앞에 두면 padding이 줄어듦. 같은 멤버인데 순서만 바꿔 12 → 8byte

손계산: S4, S5의 padding과 struct 배열 주소 (a = 0x1000)

① S4 (K = 4): c 0(끝 1) → i는 1을 4의 배수로 올려 4(끝 8) → d 8(끝 9) → 전체 9를 4의 배수로 올려 sizeof = 12. padding 1~3, 9~11 = 6byte

② S5 (K = 4): i 0(끝 4) → c 4 → d 5(끝 6) → 전체 6을 4의 배수로 올려 sizeof = 8. padding 6~7 = 2byte

③ 절약 12 − 8 = 4byte (멤버 크기는 1 + 4 + 1 = 6으로 같음). 원소 100개 배열이면 1200 vs 800byte → 400byte 차이

④ struct S2 a[10] (sizeof 24, 전체 240byte): a[1] = 0x1000 + 24 = 0x1018 → v 0x1018, i[0] 0x1020, i[1] 0x1024, c 0x1028, padding 0x1029~0x102F → 마지막 byte 0x102F = a + 47 (슬라이드와 일치) · a[3].c = 0x1000 + 24·3 + 16 = 0x1058

⑤ get_j(5) (S3, sizeof 12): leaq → %rax = 5 + 2·5 = 15 · movzwl → 주소 = a + 8 + 4·15 = 0x1000 + 68 = 0x1044. 검산: a + 12·5 + 8 = 0x1044 ✓ (a[5]는 0x103C, 거기서 j의 offset 8)

최종 답

절약이 생기는 경우: 작은 type 사이에 큰 type이 끼어 안쪽 padding이 커질 때. S1(24) → S2(24)는 순서를 바꿔도 그대로였음

구조체 배치와 정렬 — Struct Layout

char[n] 배열 길이 5 + char + short + int + long + double

+ void* + char[n]

정렬 최적화 초기화

필드 구성 — Field Order

필드	타입	크기	정렬 K	Offset	순서 변경
a	char	1 B	1	0	↑ ↓ 삭제
b	long	8 B	8	8	↑ ↓ 삭제
c	char	1 B	1	16	↑ ↓ 삭제

바이트 지도 — 8 Bytes per Row

a +0	**PAD** +1	**PAD** +2	**PAD** +3	**PAD** +4	**PAD** +5	**PAD** +6	**PAD** +7
b +8	**b** +9	**b** +10	**b** +11	**b** +12	**b** +13	**b** +14	**b** +15
c +16	**PAD** +17	**PAD** +18	**PAD** +19	**PAD** +20	**PAD** +21	**PAD** +22	**PAD** +23

```
sizeof(struct) = 24 B
최대 정렬 요구(max alignment) = 8 B
데이터 = 10 B
내부 패딩 = 7 B
끝 패딩 = 7 B

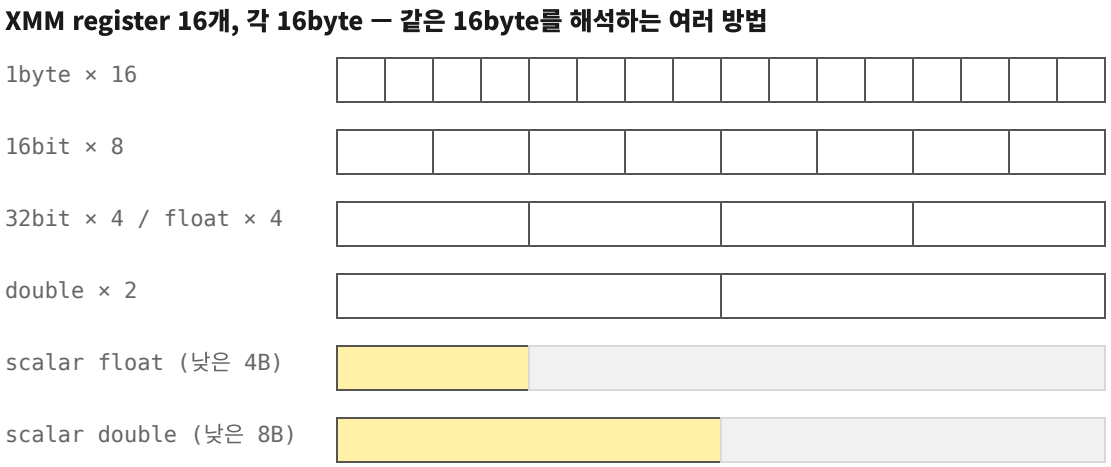
각 필드의 offset은 해당 타입 정렬 요구 K의 배수입니다.
구조체 전체 크기는 최대 정렬 요구의 배수로 올림합니다.
정렬 최적화 버튼은 휴리스틱이며 모든 경우의 최적을 보장하지 않습니다.
모델: 일반적인 x86-64 System V ABI (long, 포인터, double = 8B).
```

10 Floating point

배경: x87, SSE, AVX

- **x87 FP**: legacy 방식
- **SSE (Streaming SIMD Extensions) FP**: SIMD = Single Instruction Multiple Data. 명령어 하나로 여러 data를 한꺼번에 처리
- 오래된 machine이 지원. vector instruction의 특수한 경우로 사용
- **AVX (Advanced Vector Extensions)**는 SSE를 확장한 SIMD 명령어 집합이다. x87, SSE, AVX를 비교하면 AVX가 가장 나중에 나온 확장이지만, AVX-512와 같은 후속 확장도 존재하므로 AVX를 현재 가장 최신 버전이라고 일반화해서는 안 된다. 이 강의의 구체적인 부동소수점 명령어 예제는 슬라이드에 맞춰 SSE3 기준으로 이해한다.
- 슬라이드 32 제목이 "Programming with SSE3" → 이 수업은 SSE3 기준

XMM register



칸 수 × 크기는 모두 16byte (16×1, 8×2, 4×4, 2×8). scalar 연산은 낮은 칸 하나만 사용

---

scalar와 SIMD 연산

- `addss %xmm0,%xmm1` : scalar, single precision → 낮은 4byte 한 쌍만 더함
- `addps %xmm0,%xmm1` : SIMD, single precision → float 4개를 칸끼리 동시에 더함
- `addsd %xmm0,%xmm1` : scalar, double precision → 낮은 8byte 한 쌍만 더함

- 이름 규칙: 앞 add는 덧셈, 가운데 **s는 scalar / p는 packed(SIMD)**, 끝 **s는 single / d는 double**. AT&T 순서라 결과는 두 번째 인자 %xmm1에 저장

FP 기본: 인자와 반환

> FP 값은 XMM register로 전달: 인자는 $\%xmm0, \%xmm1, \ldots$ 순서 · 반환값은 $\%xmm0$ · XMM register는 모두 caller-saved

- callee-saved인 XMM register가 하나도 없으므로, 호출 뒤에도 값이 필요하면 호출하는 쪽이 미리 저장

```
float fadd(float x, float y) { return x + y; }
double dadd(double x, double y) { return x + y; }
```

```
# x in %xmm0, y in %xmm1
    addss %xmm1, %xmm0    # fadd: %xmm0 = x + y — 결과가 곧바로 반환 register에
    ret
    addsd %xmm1, %xmm0    # dadd: single(ss) 대신 double(sd) 명령
    ret
```

FP 메모리 참조: dincr

```
double dincr(double *p, double v) {
    double x = *p;
    *p = x + v;
    return x;
}
```

```
# p in %rdi, v in %xmm0         # 정수와 포인터는 일반 register, FP 값은 XMM register
    movapd %xmm0, %xmm1         # Copy v — 다음 movsd가 %xmm0을 덮어써 v가 사라지므로 먼저 복사
    movsd (%rdi), %xmm0         # x = *p         (memory → XMM)
    addsd %xmm0, %xmm1          # t = x + v
    movsd %xmm1, (%rdi)         # *p = t         (XMM → memory)
    ret                         # x가 %xmm0에 있어 return x를 위한 명령이 따로 없음
```

- **XMM register끼리 옮기는 mov(movapd)와 memory ↔ XMM 사이를 옮기는 mov(movsd)는 서로 다른 명령어 (movsd에는 XMM 간 이동 형식도 있으나, 여기서는 128bit 전체 복사에 movapd, 하위 double의 memory 적재/저장에 movsd 사용)**

손계산: scalar와 SIMD 덧셈, dincr 추적

① float 4칸, %xmm0 = [1.0, 2.0, 3.0, 4.0], %xmm1 = [10.0, 20.0, 30.0, 40.0] (왼쪽이 낮은 칸): addss %xmm0,%xmm1 → %xmm1 = [11.0, 20.0, 30.0, 40.0] (낮은 칸 한 쌍만 10.0 + 1.0)

② addps %xmm0,%xmm1 → %xmm1 = [11.0, 22.0, 33.0, 44.0] (네 칸 모두)

③ 둘은 각각 초기값에서 따로 실행한 결과

④ double 2칸, %xmm0 = [1.5, 9.0], %xmm1 = [2.0, 7.0]: addsd → %xmm1 = [3.5, 7.0] (낮은 8byte만 2.0 + 1.5, 위쪽 칸은 그대로)

⑤ dincr (*p = 1.5, v = 2.0): 시작 %xmm0 = 2.0, M[p] = 1.5 · movapd → %xmm1 = 2.0 · movsd → %xmm0 = 1.5 (x) · addsd → %xmm1 = 3.5 (t) · movsd → M[p] = 3.5 · ret → 반환값 %xmm0 = 1.5 (옛 *p)

최종 답 ***p는 1.5 → 3.5로 바뀌고, 반환값은 바뀌기 전 값 1.5**

11 시험 포인트

배열·포인터

- `T N[L]` 은 연속된 $L \cdot$ `sizeof(T)` byte, 원소 i의 주소 = 시작 주소 + sizeof(T) · i (char 1, short 2, int 4, long·포인터·double 8)
- 배열 이름은 첫 원소의 주소: val, val+1, &val[2], val+i는 int *이고 값은 $x, x+4, x+8, x+4i$ → 포인터에 더하는 정수는 원소 크기만큼 곱해짐
- *(val+1)은 val[1]의 값
- 범위 밖 val[5]의 값은 알 수 없음 (C 표준상 UB)
- Intel은 little endian: int 3은 03 00 00 00으로 저장
- `addl $1, (%rdi,%rax,4)` = z[i]++: base = 배열 시작, index = i, scale = 원소 크기, 메모리 값을 직접 갱신
- for (i = 0; i < 5; i++)는 `cmpq $4, %rax` + `jbe` 로 컴파일됨 (size_t는 unsigned 비교)

다차원·다단계 배열

- `T N[R][C]` : 크기 $R \cdot C \cdot k$, row-major 배치, 한 행은 $C \cdot k$ byte
- 행 시작 주소 = $N + i \cdot (C \cdot k)$ · 원소 주소 = $N + (i \cdot C + j) \cdot k$
- `leaq (%rdi,%rdi,4)` = 5·index (scale은 1, 2, 4, 8뿐이라 x + 4x로 5배), 이어지는 leaq/movl에서 원소 크기 4를 곱함
- int *univ[3] 같은 multi-level 배열은 원소 접근에 메모리 읽기 두 번: `Mem[Mem[univ+8·index]+4·dig]` (행 포인터 → 원소)
- nested는 주소 계산 후 한 번만 읽음
- multi-level의 행은 서로 떨어져 있을 수 있음 (예: 16, 56, 36)
- nested는 행 간격이 항상 $C \cdot k$
- 고정 크기 int[16][16] 접근은 `salq $6` (64·i) + addq + movl
- 가변 크기 행렬의 주소: int a[n][n]에서 원소 a[i][j]의 주소는 a + 4·n·i + 4·j이다. 먼저 `imulq` 로 행 번호와 열 개수의 곱인 n·i를 계산한다.

- 명령어 선택: `leaq` 로 행 시작 주소 a + 4·n·i를 계산하고, `movl` 로 4·j만큼 떨어진 위치의 원소를 읽는다. 행 크기가 컴파일 시점에 상수이면 행 offset의 곱셈을 shift로 대체할 수 있지만, 행 크기가 실행 중 결정되면 일반적으로 `imulq` 가 필요하다.

---

struct·alignment

- 구조체 field는 선언 순서대로, offset은 컴파일 시점에 정해짐
- struct rec은 offset 0 / 16 / 24, 크기 32 · &r->a[idx]는 leaq 한 줄
- 정렬: 기본 type이 B byte를 요구하면 주소는 B의 배수
- 구조체는 K = field 중 가장 큰 정렬 요구. 시작 주소와 전체 길이가 K의 배수가 되도록 compiler가 internal padding과 끝 padding을 넣음
- struct S1 {char c; int i[2]; double v;} : offset 0, 4, 16, sizeof 24 (padding 3 + 4)
- 순서를 바꾼 S2 {double v; int i[2]; char c;} : offset 0, 8, 16, 끝 padding 7 → 크기는 똑같이 24
- 구조체 배열 원소 a[idx]의 주소 = a + sizeof·idx (padding 포함 크기), member는 거기에 offset을 더함
- S3는 sizeof 12, j의 offset 8 → `leaq (%rdi,%rdi,2)` (3·idx) + `movzwl a+8(,%rax,4)`
- 공간 절약은 큰 type을 앞에: S4 {char c; int i; char d;} 12byte → S5 {int i; char c; char d;} 8byte

---

Floating point

- FP: XMM register는 16개, 각 16byte · 인자는 %xmm0, %xmm1, … · 반환은 %xmm0 · 전부 caller-saved
- addss·addsd는 scalar(낮은 4 / 8byte 한 쌍), addps는 SIMD(float 4개 동시)
- FP 값은 XMM register, 정수와 포인터 인자는 일반 register (dincr는 p가 %rdi, v가 %xmm0)
- 이동 명령어가 다름: XMM끼리는 movapd, memory ↔ XMM은 movsd (movsd에는 XMM 간 이동 형식도 있음)

12 연습문제

문제 1. double d[5]의 시작 주소가 0x500일 때 ① 전체 크기 ② &d[3] ③ d+4 ④ 마지막 원소 d[4]가 차지하는 주소 범위를 구하라.

정답 보기

① 전체 크기 = 5 × 8 = 40 = 0x28 byte

② &d[3] = 0x500 + 3 × 8 = 0x500 + 0x18 = 0x518

③ d+4 = 0x500 + 4 × 8 = 0x500 + 0x20 = 0x520 (double *에 더하므로 원소 크기 8을 곱함)

④ d[4]는 0x520~0x527 (8byte), 배열 전체는 0x500~0x527

최종 답 **① 0x28 ② 0x518 ③ 0x520 ④ 0x520~0x527**

문제 2. int w[6] = {2, 4, 6, 8, 10, 12}이고 w의 시작 주소가 100(십진수)일 때 w[3], *(w+4), &w[5], w+2, w[6]의 type과 값(또는 주소)은?

정답 보기

① w[3]: int, 8

② *(w+4): int, 10

③ &w[5]: int *, 100 + 4 × 5 = 120

④ w+2: int *, 100 + 4 × 2 = 108

⑤ w[6]: int이지만 범위 밖 (주소 100 + 24 = 124) → 값을 알 수 없음 (C 표준상 UB)

최종 답 **8 · 10 · 120 · 108 · 알 수 없음**

문제 3. zincr 어셈블리의 cmpq $4, %rax + jbe .L4에서

- ① i = 5일 때 점프하지 않는 이유를 CF, ZF로 설명하고,
- ② ZLEN이 8이면 비교 상수가 무엇으로 바뀌는지,
- ③ zincr(postech)를 연속 두 번 호출하면 postech이 어떻게 되는지 구하라.

정답 보기

① 5 − 4 = 1 → unsigned로 borrow 없음(CF = 0), 결과가 0이 아님(ZF = 0) → jbe는 CF = 1 또는 ZF = 1일 때만 점프하므로 점프 안 함, 함수 종료

② 마지막 index가 7이므로 cmpq $7, %rax

③ 3 7 6 7 3 → 4 8 7 8 4 → 5 9 8 9 5

최종 답 **① CF = 0, ZF = 0이라 jbe 불성립 ② cmpq $7 ③ 5 9 8 9 5**

문제 4. int M[6][4]의 시작 주소가 0x1000일 때 ① 전체 크기 ② 한 행의 크기 ③ M[3]의 시작 주소 ④ &M[4][2]를 두 가지 식으로 계산하라.

정답 보기

① 전체 크기 = 6 × 4 × 4 = 96 = 0x60 byte

② 한 행 = 4 × 4 = 16 = 0x10 byte

③ M[3] = 0x1000 + 3 × 16 = 0x1000 + 0x30 = 0x1030

④ 식 ① $M + i \cdot (C \cdot k) + j \cdot k$ = 0x1000 + 4 × 16 + 2 × 4 = 0x1000 + 0x48 = 0x1048 · 식 ② $M + (i \cdot C + j) \cdot k$ = 0x1000 + (4 × 4 + 2) × 4 = 0x1000 + 72 = 0x1048 → 일치

최종 답 **① 0x60 ② 0x10 ③ 0x1030 ④ 0x1048**

문제 5. long L[3][5]의 시작 주소가 0x2000이고 `long get(long i, long j) { return L[i][j]; }`가 아래처럼 컴파일되었다. 빈칸을 채우고 get(2, 4)가 읽는 주소를 구하라.

```
# i in %rdi, j in %rsi
    leaq (%rdi,%rdi,4), %rax   # (a)
    addq %rax, %rsi            # (b)
    movq L(,__,__), %rax       # (c)
    ret
```

정답 보기

① (a) 5·i (열 개수가 5) · (b) 5·i + j · (c) L(,%rsi,8) → M[L + 8·(5·i + j)]. long이라 scale이 8

② get(2, 4): %rax = 2 + 4·2 = 10 → %rsi = 4 + 10 = 14 → 주소 0x2000 + 8 × 14 = 0x2000 + 112 = 0x2070

③ 검산: 행 2의 시작 0x2000 + 2 × (5 × 8) = 0x2050, 거기에 4 × 8 = 32 = 0x20을 더하면 0x2070 ✓

최종 답 **(a) 5i (b) 5i + j (c) L(,%rsi,8) · 주소 0x2070**

문제 6. get_pohang_digit(3, 2)를 실행할 때 각 명령어 직후의 %rax, %rsi와 읽는 주소, 반환값을 구하라 (pohang = 76).

정답 보기

① leaq (%rdi,%rdi,4), %rax → %rax = 3 + 4·3 = 15

② addq %rax, %rsi → %rsi = 2 + 15 = 17 (슬라이드 표기 addl — index·col이 32bit int라 5·index+col이 32bit 범위를 넘지 않는 이 예에서는 addq와 결과 동일; 32bit 쓰기는 상위 32bit를 0으로 만듦)

③ movl pohang(,%rsi,4), %eax → 주소 76 + 4 × 17 = 76 + 68 = 144

④ 검산: 행 3의 시작 136 + 2 × 4 = 144, 그 행은 3 7 6 7 4 → 세 번째 값 → %eax = 6

최종 답 **%rax = 15, %rsi = 17, 주소 144, 반환값 6**

문제 7. int A[3][2] = {{1, 2}, {3, 4}, {5, 6}}을

- ① row-major로 저장했을 때 메모리 순서,
- ② A[2][1]의 시작 주소로부터의 offset(byte)과 값,
- ③ column-major였다면 메모리 순서를 쓰라.

정답 보기

① row-major: 1 2 3 4 5 6

② offset = $(2 \cdot 2 + 1) \cdot 4 = 20$ byte → flat index 5, 값 6

③ column-major: 첫 열 1 3 5, 둘째 열 2 4 6 → 1 3 5 2 4 6

최종 답 **① 1 2 3 4 5 6 ② offset 20, 값 6 ③ 1 3 5 2 4 6**

문제 8. int *tab[3]의 시작 주소가 0x100이고 tab[0] = 0x400, tab[1] = 0x500, tab[2] = 0x440이다.

- 0x400부터 1 2 3 4
- 0x440부터 9 8 7 6
- 0x500부터 5 5 5 5 (int로 저장)

`int get(size_t index, size_t col) { return tab[index][col]; }`가 슬라이드의 get_univ_digit처럼 컴파일되었을 때 get(2, 3)에 대해 다음을 구하라.

- ① 첫 번째 메모리 읽기의 주소와 읽은 값
- ② 최종 주소
- ③ 반환값
- ④ 같은 값들이 int t[3][4](시작 0x400, row-major)에 들어 있는 nested 배열이었다면 t[2][3]의 주소를 구하라.

정답 보기

① 첫 번째 읽기: tab + 8·2 = 0x100 + 16 = 0x110 → 읽은 값(행 포인터) 0x440

② salq $2로 col × 4 = 12 = 0xC → 최종 주소 0x440 + 0xC = 0x44C

③ 0x440부터 9 8 7 6이므로 셋째(index 3) 값 6. 메모리 읽기는 두 번

④ nested였다면 0x400 + 2 × (4 × 4) + 3 × 4 = 0x400 + 32 + 12 = 0x42C. 읽기는 한 번

최종 답 **① 0x110에서 0x440 ② 0x44C ③ 6 ④ 0x42C**

문제 9. `long var_ele(size_t n, long a[n][n], long i, long j) { return a[i][j]; }`가 아래처럼 컴파일되었다.

빈칸을 채우고, n = 7, a = 0x3000, i = 4, j = 2일 때 읽는 주소를 구하라.

또 n이 상수 8인 long fix[8][8]이면 행 시작 offset을 만드는 명령은 무엇이 되는가?

```
# n in %rdi, a in %rsi, i in %rdx, j in %rcx
    imulq %rdx, %rdi           # (a)
    leaq (%rsi,%rdi,__), %rax  # (b)
    movq (%rax,%rcx,__), %rax  # (c)
    ret
```

정답 보기

① (a) n·i · (b) scale 8, a + 8·n·i · (c) scale 8, a + 8·n·i + 8·j. long이라 원소 크기 8

② n·i = 7 × 4 = 28 → 0x3000 + 8 × 28 = 0x3000 + 224 = 0x30E0 → 열 offset 8 × 2 = 16을 더해 0x30F0

③ 검산: (4 × 7 + 2) × 8 = 240 = 0xF0 → 0x3000 + 0xF0 = 0x30F0 ✓

④ fix[8][8]은 행 하나가 8 × 8 = 64byte = 2^6 → 곱셈 대신 salq $6, %rsi (64·i)

최종 답 **(a) n·i (b) 8 (c) 8 · 주소 0x30F0 · 상수면 salq $6**

문제 10. struct T { short s; char c; double d; int i; };에서

- ① 각 member의 offset과 sizeof
- ② padding 총량
- ③ member 순서를 d, i, s, c로 바꾼 구조체의 offset과 sizeof, 줄어든 크기를 구하라.

정답 보기

① K = 8 (double). s 0 (끝 2), c 2 (끝 3), d는 3을 8의 배수로 올려 8 (끝 16), i 16 (끝 20), 전체 20을 8의 배수로 올려 sizeof = 24

② member 크기 2 + 1 + 8 + 4 = 15 → padding 24 − 15 = 9byte (3~7, 20~23)

③ 순서 변경: d 0 (끝 8), i 8 (끝 12), s 12 (끝 14), c 14 (끝 15), 전체 15를 8의 배수로 올려 sizeof = 16 → 24 − 16 = 8byte 감소 (padding 1byte뿐)

최종 답 **① 0, 2, 8, 16 / sizeof 24 ② 9byte ③ 0, 8, 12, 14 / sizeof 16, 8byte 감소**

문제 11. struct P { char tag; double val; short n; } arr[20];가 있고 arr의 시작 주소는 0x5000이다.

- ① sizeof(struct P)와 arr 전체 크기
- ② &arr[7].n
- ③ `short get_n(int idx) { return arr[idx].n; }`가 아래처럼 컴파일되었을 때 빈칸 두 개와 %rax의 값(idx = 7)을 구하라.

```
# idx in %rdi
    leaq (%rdi,%rdi,2), %rax
    movzwl arr+__(,%rax,__), %eax
    ret
```

정답 보기

① K = 8. tag 0, val 8, n 16 (끝 18) → 18을 8의 배수로 올려 sizeof = 24, 전체 24 × 20 = 480 = 0x1E0 byte

② &arr[7].n = 0x5000 + 24 × 7 + 16 = 0x5000 + 168 + 16 = 0x50B8

③ n의 offset이 16이므로 첫 빈칸 16. %rax = 3·idx에 곱해 24·idx가 되려면 scale 8 → movzwl arr+16(,%rax,8). idx = 7이면 %rax = 7 + 2 × 7 = 21, 주소 0x5000 + 16 + 8 × 21 = 0x50B8. ②와 일치 ✓

최종 답 **① sizeof 24, 전체 0x1E0 ② 0x50B8 ③ arr+16(,%rax,8), %rax = 21**

문제 12.

- ① `long f(long n, double x, float y, double *p)`의 네 인자는 각각 어느 register로 전달되는가?
- ② dincr가 *p = 3.25, v = 0.75에서 실행될 때 movapd, movsd (%rdi), %xmm0, addsd, movsd %xmm1, (%rdi) 직후의 값과 반환값을 구하라.
- ③ %xmm0 = [1.5, 2.5, 3.5, 4.5], %xmm1 = [0.5, 0.5, 0.5, 0.5](float 4칸)일 때 addps %xmm0,%xmm1과 addss %xmm0,%xmm1을 각각 실행한 뒤의 %xmm1은?

정답 보기

① 정수와 포인터는 일반 register 순서, FP는 XMM register 순서로 따로 셈: n → %rdi, p → %rsi, x → %xmm0, y → %xmm1

② movapd → %xmm1 = 0.75 · movsd → %xmm0 = 3.25 · addsd → %xmm1 = 0.75 + 3.25 = 4.0 · movsd → M[p] = 4.0 · 반환값 %xmm0 = 3.25 (옛 *p)

③ addps: 네 칸 모두 더해 [2.0, 3.0, 4.0, 5.0] · addss: 낮은 칸만 1.5 + 0.5 = 2.0이 되고 나머지는 그대로 [2.0, 0.5, 0.5, 0.5]

최종 답

① n→%rdi, p→%rsi, x→%xmm0, y→%xmm1 ② *p = 4.0, 반환 3.25 ③ [2,3,4,5] / [2,0.5,0.5,0.5]